Agentic Research Agent

Enterprise architecture and technical design document for a provider-agnostic LangChain and LangGraph research assistant with RAG, web search, calculator tooling, typed configuration, CLI entry points, and automated tests.

Document type: Architecture Review | Generated: 2026-06-04 | Runtime: Python 3.13

Package: agentic_research_agent

Executive Summary

The project is structured as a small but credible enterprise reference implementation. The code separates configuration, model construction, orchestration, tools, schemas, and CLI delivery. The agent uses an explicit LangGraph ReAct loop, which makes control flow auditable and testable.

Strength

Clear layered package layout and a public service facade through `ResearchAgent`.

Strength

Provider-agnostic LLM factory isolates concrete SDK choices from orchestration code.

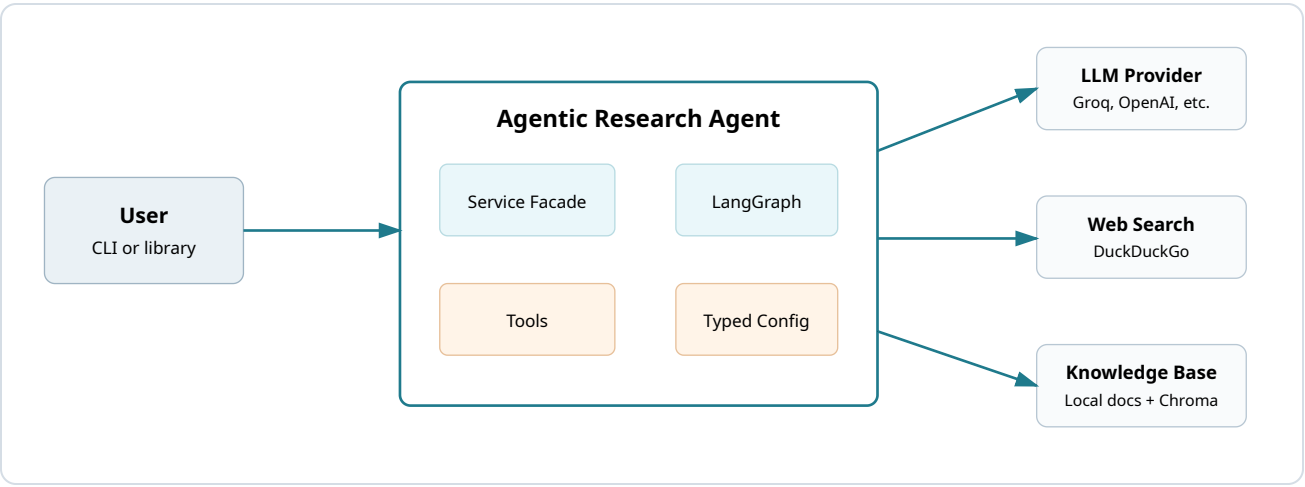
Improved

Added explicit runtime dependencies, stricter RAG configuration validation, structured run metadata, production JSON logging, and calculator resource limits.

Next maturity step

For production deployment, replace in-memory checkpointing with durable storage and add centralized tracing, auth, and deployment manifests.

System Context

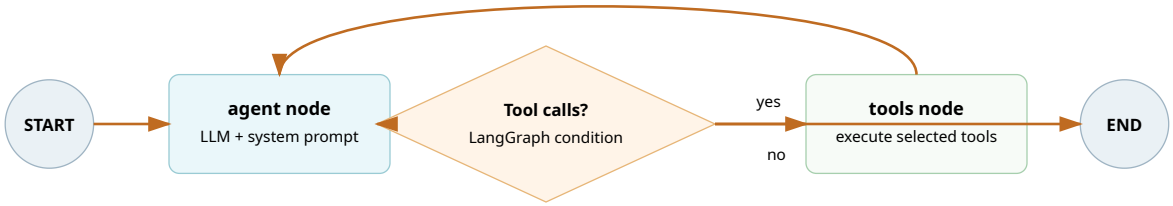


The application boundary is clean: external systems are accessed only through dedicated infrastructure/tool modules.

Layered Architecture

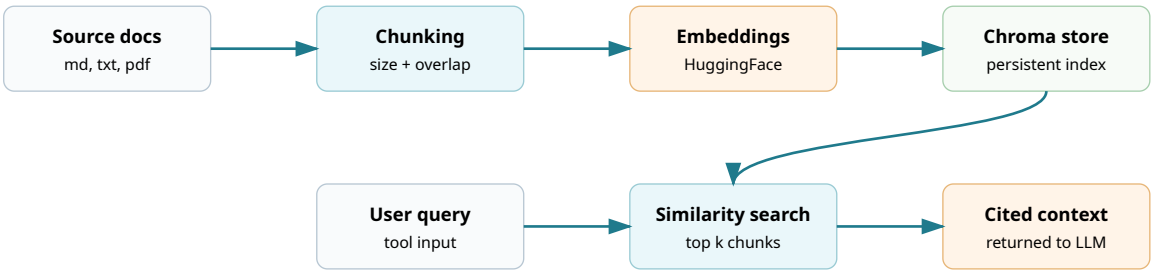
Layer	Primary modules	Responsibility	Enterprise concern
Delivery	`cli.py`, library import	Receives user input and renders answers.	Keep UI concerns out of agent orchestration.
Application service	`agents/service.py`	Builds dependencies and exposes `ask` and `stream`.	Stable boundary for future API, workers, or schedulers.
Orchestration	`agents/graph.py`, `state.py`, `prompts.py`	Defines ReAct loop, state reducer, and system behavior.	Auditable control flow and deterministic test surface.
Capabilities	`tools/*`	Calculator, web search, and RAG retrieval.	Each tool can be secured, tested, and monitored independently.
Infrastructure	`core/llm.py`, `core/logging.py`, `config/settings.py`	Provider factory, logging, and environment-driven settings.	Deployment portability and operational control.
Contracts	`schemas/models.py`	Pydantic request and response models.	API-ready validation and serialization.

Agent Control Flow



The graph is intentionally explicit: the model decides whether to answer or request tools, `ToolNode` executes requested tools, and the result returns to the model until a final answer is produced.

RAG Data Flow



Ingestion is separated from query execution. Documents are loaded from `data/knowledge_base`, chunked, embedded locally, and persisted under `data/vector_store`. Query-time retrieval returns source filenames with excerpts so final answers can cite evidence.

Technical Details

Language and packaging

Python 3.13, `pyproject.toml`, Hatchling build backend, console script `research-agent`.

Agent framework

LangChain chat model interface plus LangGraph `StateGraph` and `ToolNode`.

Configuration

Pydantic Settings reads environment variables and optional `.env` file.

Retrieval

Chroma vector store with local HuggingFace embeddings and recursive character splitting.

Observability

Central logging, production JSON formatter, run id, duration, thread id, and tool call count.

Quality

Offline pytest suite covers graph routing, configuration validation, calculator safety, and provider factory errors.

Configuration Matrix

Category	Variables	Purpose
Application	`ENVIRONMENT`, `LOG_LEVEL`	Controls environment-specific behavior and log verbosity.
LLM	`LLM_PROVIDER`, `LLM_MODEL`, `LLM_TEMPERATURE`, `LLM_MAX_TOKENS`, `LLM_TIMEOUT_SECONDS`	Selects model provider and inference behavior.
Credentials	`GROQ_API_KEY`, `ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `OLLAMA_BASE_URL`	Provider-specific connection details. Secrets must remain outside source control.
RAG	`EMBEDDING_MODEL`, `KNOWLEDGE_BASE_DIR`, `VECTOR_STORE_DIR`, `CHUNK_SIZE`, `CHUNK_OVERLAP`, `RETRIEVER_TOP_K`	Controls ingestion, vector storage, and retrieval quality.
Agent behavior	`MAX_SEARCH_RESULTS`, `RECURSION_LIMIT`	Bounds external search volume and graph execution depth.

Security and Reliability Review

Area	Current control	Recommendation
Secrets	Environment variables and `.env` exclusion.	Use a managed secret store in staging and production.
Tool execution	Calculator uses AST allow-list, no `eval`, with length and complexity limits.	Add policy checks before introducing tools that perform writes or network mutations.
External search	DuckDuckGo errors are caught and returned as recoverable tool text.	Add retry/backoff and provider abstraction for production search SLAs.
Memory	In-process `MemorySaver` keeps thread context during process lifetime.	Use durable checkpointer such as Postgres or SQLite for long-lived sessions.
Observability	Run id, duration, tool count, and JSON logs in production.	Integrate OpenTelemetry or LangSmith traces with alerting dashboards.
Data governance	Local RAG documents and generated vector store are separated.	Add document provenance, retention policy, and PII scanning before indexing sensitive corpora.

Deployment View

The current project is CLI-first, but the service layer is already suitable for a FastAPI handler, queue consumer, or scheduled batch worker because the outer delivery mechanism only needs to call `ResearchAgent.ask()` or `ResearchAgent.stream()`.

Implemented Review Changes

- Added explicit package dependencies for `pydantic-settings` and `typer`.
- Added validation that `CHUNK_OVERLAP` must be smaller than `CHUNK_SIZE`.
- Added production JSON logging support while preserving Rich logs for local development.
- Added `run_id` and `duration_ms` to agent responses for observability.
- Added calculator length, AST complexity, and exponent magnitude guards.
- Added tests for retrieval config validation and calculator resource limits.

Validation Evidence

Check	Result	Notes
`uv run pytest`	**Passed**	30 tests passed.
`uv run ruff check .`	**Passed**	No lint violations found.

Production Readiness Roadmap

1. Expose the service through FastAPI with authentication, request validation, rate limits, and health checks.
2. Move checkpointing from memory to a durable backend and define conversation retention rules.
3. Add OpenTelemetry or LangSmith tracing for model calls, tool calls, latency, and failures.
4. Introduce CI with pytest, Ruff, mypy, dependency audit, and build verification.
5. Containerize the application and define separate dev, staging, and production configuration profiles.
6. Add RAG evaluation: retrieval precision, answer faithfulness, citation quality, and regression datasets.